

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа 4

Выполнил:

Лошкарёв Д. В.

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Цель работы:

Переход от монолитной архитектуры приложения «Сервис для обмена рецептами» к микросервисной архитектуре.

Основные задачи:

1. Разделение бизнес-логики на независимые компоненты (микросервисы).
2. Реализация принципа Database-per-service (у каждого сервиса своя база данных).
3. Проектирование межсервисного взаимодействия (синхронного через REST API).
4. Описание эндпоинтов для внутреннего взаимодействия в формате OpenAPI.
5. Обеспечение отказоустойчивости и независимого развертывания модулей системы.

Ход работы

1. Анализ и декомпозиция монолитного приложения

На основе существующей структуры проекта «Сервис для обмена рецептами» было принято решение разделить монолитное приложение на три независимых микросервиса. Это разделение продиктовано логической обособленностью данных и различными требованиями к масштабируемости: сервис рецептов будет испытывать высокую нагрузку на чтение, в то время как сервис взаимодействий — на запись.

Выделенные микросервисы:

- Auth Service: управление пользователями, аутентификация и социальные связи (подписки).

- Recipe Service: управление каталогом рецептов, ингредиентами и шагами приготовления.
- Interaction Service: система лайков, комментариев и избранного.

2. Проектирование баз данных в концепции Database-per-service

Каждый сервис получает собственную изолированную базу данных. Прямые JOIN-запросы между таблицами разных сервисов запрещены. Связи реализуются на уровне бизнес-логики через хранение идентификаторов (ID).

Схема таблиц микросервиса Auth Service (База данных db_auth):

Таблица users:

- id (Primary Key)
- username (varchar)
- email (varchar, unique)
- password_hash (varchar)
- bio (text)
- avatar_url (varchar)
- created_at (timestamp)

Таблица follows (связь многие-ко-многим):

- follower_id (int, ссылка на пользователя)
- following_id (int, ссылка на того, на кого подписались)

Схема таблиц микросервиса Recipe Service (База данных db_recipes):

Таблица recipes:

- id (Primary Key)
- title (varchar)
- description (text)
- dish_type (varchar)
- difficulty (varchar)

- video_url (varchar)
- author_id (int, логическая ссылка на пользователя из другого сервиса)
- created_at (timestamp)

Таблица ingredients:

- id (Primary Key)
- name (varchar, unique)

Таблица recipe_ingredients:

- id (Primary Key)
- recipe_id (int, Foreign Key внутри сервиса)
- ingredient_id (int, Foreign Key внутри сервиса)
- amount (varchar)

Таблица recipe_steps:

- id (Primary Key)
- recipe_id (int, Foreign Key внутри сервиса)
- step_number (int)
- instruction (text)
- image_url (varchar)

Схема таблиц микросервиса Interaction Service (База данных db_interactions):

Таблица likes:

- id (Primary Key)
- user_id (int, логическая ссылка на пользователя)
- recipe_id (int, логическая ссылка на рецепт)

Таблица comments:

- id (Primary Key)
- user_id (int, логическая ссылка)

- recipe_id (int, логическая ссылка)
- content (text)
- created_at (timestamp)

Таблица favorites:

- id (Primary Key)
- user_id (int, логическая ссылка)
- recipe_id (int, логическая ссылка)
-

3. Проектирование межсервисного взаимодействия

Взаимодействие между сервисами будет осуществляться по протоколу HTTP (REST API). Для обеспечения целостности данных и формирования полных ответов для фронтенда спроектированы внутренние (Internal) эндпоинты, которые не доступны пользователям извне напрямую, а используются только микросервисами.

Сценарий взаимодействия: Когда клиент запрашивает информацию о рецепте через Recipe Service, сервис выполняет следующие действия:

1. Запрашивает данные о рецепте из своей БД.
2. Делает запрос к Auth Service для получения публичного профиля автора по author_id.
3. Делает запрос к Interaction Service для получения счетчика лайков и списка комментариев по recipe_id.
4. Агрегирует данные в один ответ и возвращает клиенту.
5. Описание эндпоинтов внутреннего взаимодействия (Internal API)

Для реализации этой схемы необходимо внедрить следующие эндпоинты:

Сервис Auth Service: GET /internal/users/{id}

- Описание: Получение базовой информации о пользователе (имя, аватар) для отображения в рецептах или комментариях.
- Ответ 200: { id, username, avatar_url }
- Ошибка 404: Пользователь не найден.

Сервис Recipe Service: GET /internal/recipes/{id}/exists

- Описание: Проверка существования рецепта. Используется Interaction Service перед сохранением лайка или комментария.
- Ответ 200: { exists: true }
- Ошибка 404: Рецепт не существует.

Сервис Interaction Service: GET /internal/stats/recipe/{id}

- Описание: Получение агрегированной статистики по рецепту.
- Ответ 200: { recipe_id, likes_count, comments_count }

Данная архитектура позволяет каждому сервису развиваться независимо и использовать собственные настройки БД, сохраняя при этом общую функциональность приложения.

Вывод

В ходе выполнения ДЗ-4 я разделил монолитное приложение на три микросервиса: Auth, Recipe и Interaction. Основным изменением стал переход на архитектуру Database-per-service, где у каждого сервиса теперь своя изолированная база данных.

Я продумал, как сервисы будут общаться между собой через внутренние API эндпоинты, так как теперь нельзя использовать обычные JOIN-запросы между таблицами разных сервисов. Это позволило сохранить целостность данных и возможность собирать полную информацию для пользователя.

Такой подход делает систему более надежной: если один сервис (например, комментарии) временно упадет, основная часть приложения с рецептами продолжит работать. В итоге я подготовил технический проект для реализации микросервисов в следующей лабораторной работе.